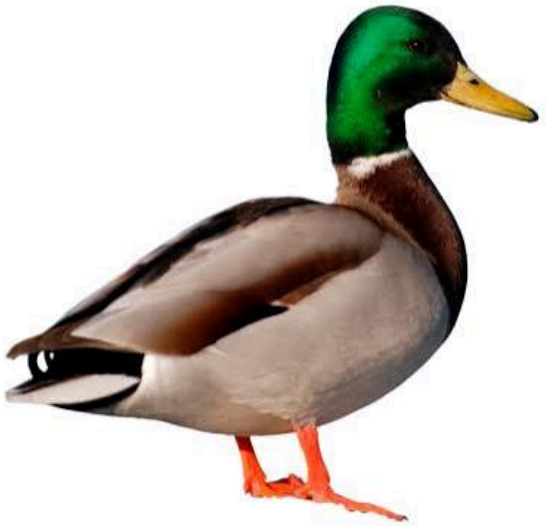
Ciência de Dados Aplicada ao Direito I

Aula 20

Modelos de linguagem



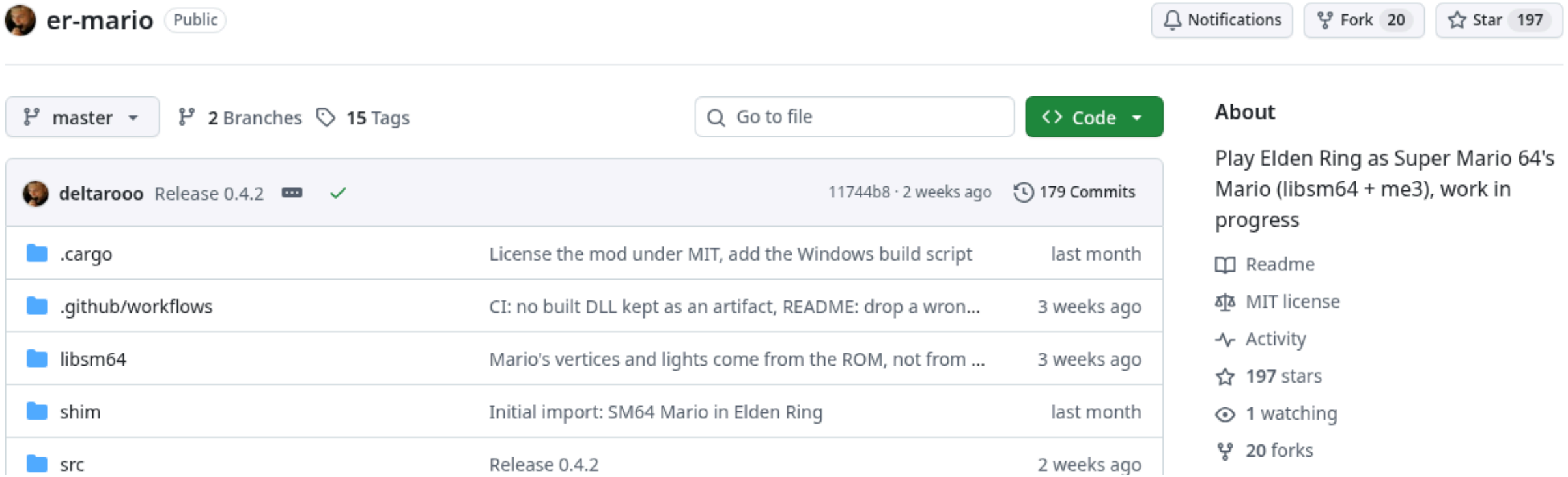
Prof. Bruno da Cunha de Oliveira - 2026/2

O Mario em outros jogos

O Mario em outros jogos

Animação no navegador: `animacao-video-mario.html`.

Mario no Elden Ring, setembro de 2026



Fonte: github.com/deltarooo/er-mario, 26/10/2026

Posso vender?

O repositório do mod do Elden Ring tem licença MIT, que deixa copiar, mudar e vender.

Posso pegar esse Mario, pôr num jogo meu e vender o jogo? Cada um escreve sim ou não, e por quê.

O que a MIT do mod cobre

RESPOSTA

> *"MIT. This covers the mod's own code only, not Super Mario 64's or Elden Ring's content: Mario's model, textures, sounds and animations come from the player's ROM."*

A licença só dá o que é do autor: o código que ele escreveu. O Mario não é dele. Cada jogador põe na pasta a ROM do próprio cartucho, e o modelo, as texturas e os sons saem de lá.

Mas o movimento do Mario é código

O mod usa um pacote chamado `libsm64`, que se descreve como "mostly code from the decompilation project".

Em 2019, um grupo leu o programa do Super Mario 64, que só existe compilado no cartucho, e o reescreveu em C até que, compilado de novo, desse o mesmo arquivo, byte a byte. Isso é decompilar.

O resultado está no GitHub desde 08/06/2019, com licença de domínio público (CC0). Sem os desenhos e os sons: para compilar, cada um precisa da própria ROM.

A Nintendo deixa?

O código do Super Mario 64, reconstruído a partir do cartucho, está aberto no GitHub há sete anos.

A Nintendo deixa isso no ar? Cada um escreve sim ou não, e por quê.

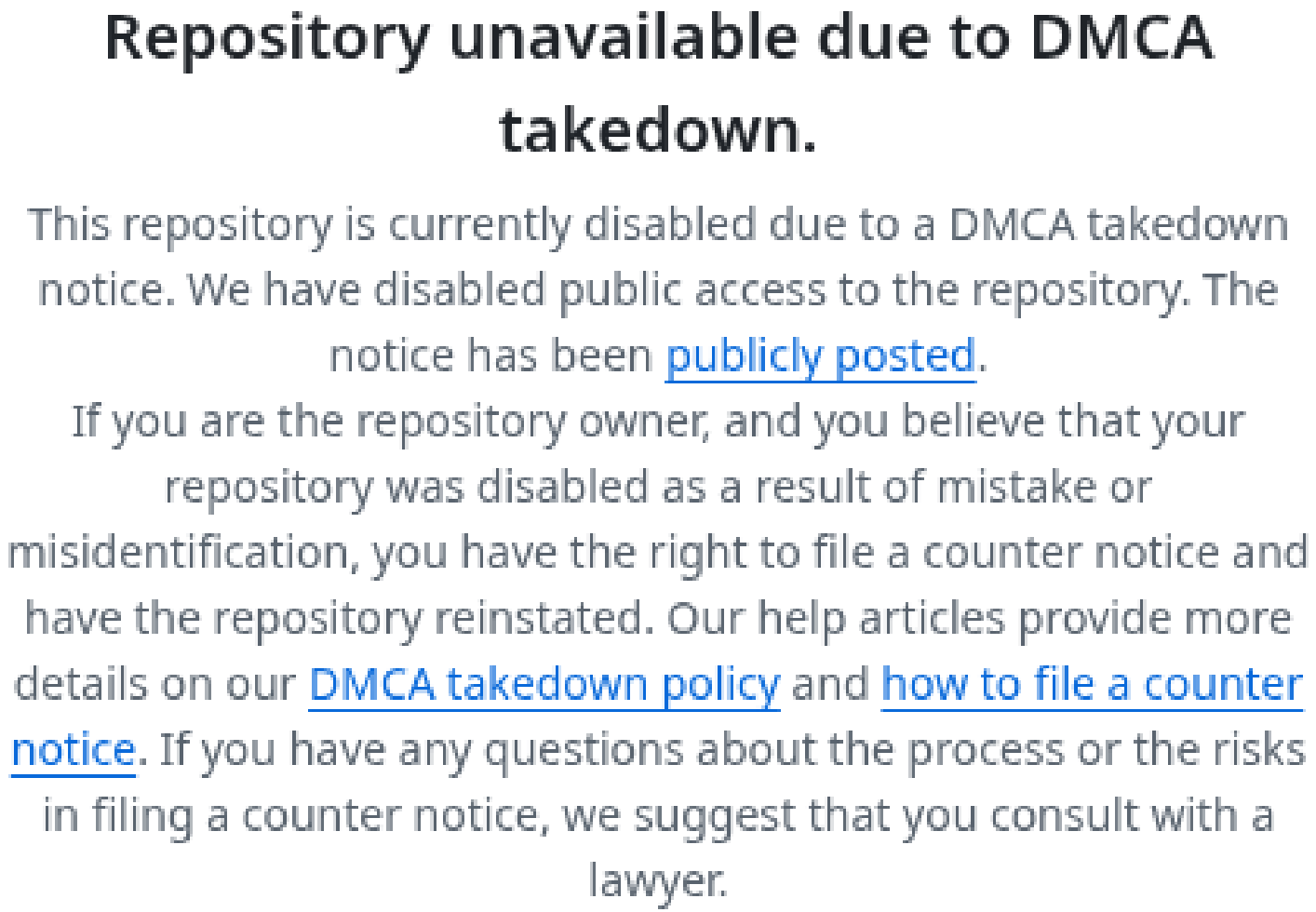
Deixou, até agora

RESPOSTA

O Super Mario 64 decompilado segue no ar. Mas em 23/10/2026 a Nintendo notificou o GitHub contra a decompilação de outro jogo dela, o Rhythm Tengoku, e 21 cópias, por distribuírem "decompiled copies" do código e dos desenhos e sons do jogo.

O GitHub bloqueou o repositório no mesmo dia.

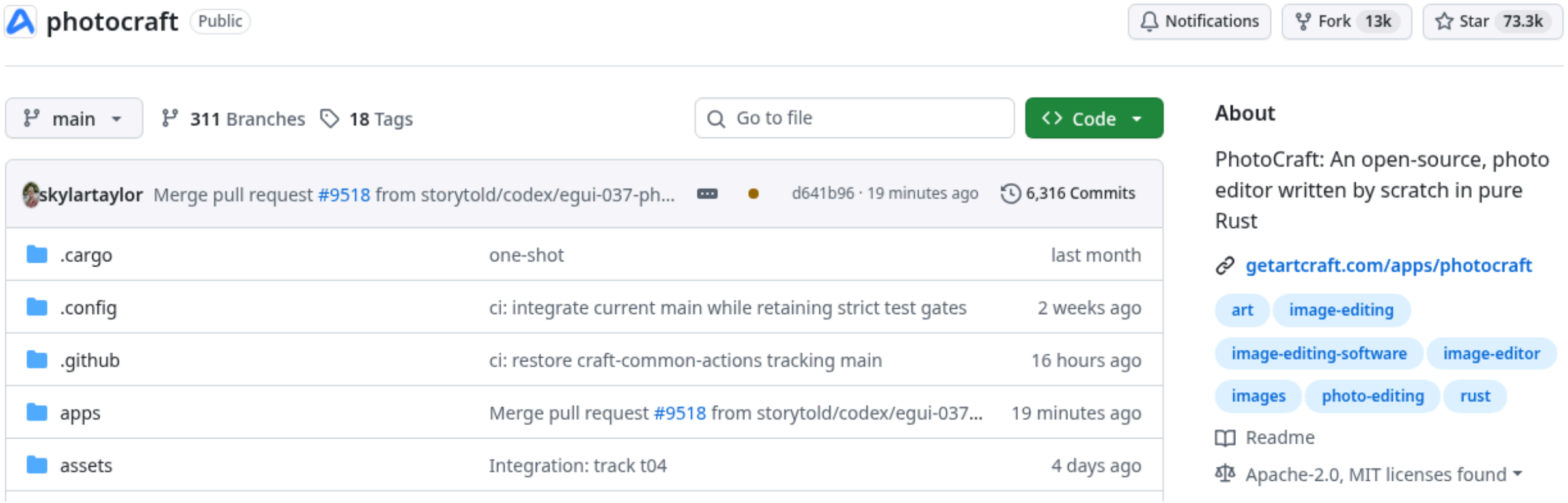
Rhythm Tengoku, decompilação bloqueada em 23/10/2026



Fonte: github.com/arthurtilly/rhythmtengoku, 26/10/2026

E se ninguém olhar o código?

PhotoCraft, outubro de 2026



Fonte: github.com/storytold/photocraft, 26/10/2026

Um Photoshop aberto, feito com IA

Um programador, Brandon Thomas, publicou entre 30/09 e 01/10/2026 sete programas que imitam os da Adobe, como Photoshop, Illustrator e Premiere, escritos com o Claude. O de imagem, o PhotoCraft, tinha 73 mil estrelas no GitHub em 26/10.

> *"Clean-room: implemented from public specs and observed behaviour only. No proprietary code, shaders or assets."*

Licença: MIT ou Apache-2.0, à escolha de quem usa.

Clean room, desde os anos 1980

Para vender computadores compatíveis com o PC da IBM sem copiar o programa de inicialização dela, a BIOS, empresas como a Compaq e a Phoenix dividiram o trabalho em duas equipes.

A primeira estuda o original e escreve uma especificação, sem código. A segunda, que nunca viu o original, programa a partir do texto. Se o resultado parecer com o da IBM, a defesa é que foi criado de forma independente.

A Compaq lançou o seu em 1982; a Phoenix passou a licenciar a BIOS dela em 1984. Quem copiou o código da IBM, como a Eagle e a Corona, fez acordo e reescreveu.

Quem está mais perto de copiar?

De um lado, o Mario decompilado: o código da Nintendo, lido no cartucho e reescrito até ficar idêntico.

Do outro, o PhotoCraft: o comportamento do Photoshop, observado e reescrito sem ver o código.

Qual dos dois está mais perto de copiar? Cada um escreve a resposta e o porquê.

A decompilação é o contrário do clean room

RESPOSTA

A decompilação lê o código do dono e reconstrói o mesmo programa. O clean room não lê o código e reconstrói só o comportamento. O PhotoCraft diz estar do segundo lado, mas ninguém de fora conferiu, e o programador que "nunca viu o original" é um modelo treinado com código cujo conteúdo ninguém de fora conhece.

E até 10/10/2026 o pacote principal do ArtCraft trazia efeitos sonoros da Nintendo. A troca para MIT ou Apache os deixou de fora: "assets that ArtCraft doesn't own".

A licença de vocês

Na aula passada, vimos que o grupo escolhe uma licença para o repositório. Ela cobre o que o grupo escreveu.

O resto tem dono e licença próprios: as decisões raspadas do tribunal, as bibliotecas instaladas com `uv add` e, a partir de hoje, o modelo de linguagem que o programa vai chamar.

Quem lembra vale ponto

Insper

O que significam estas palavras?

Deblaterar

Estugar

Escrevam o significado de cada uma. Cada palavra certa vale 0,1 ponto extra.

Deblaterar

falar com veemência contra alguém ou algo; bradar

Estugar

apressar o passo; estimular, incitar

Os dois vocabulários do resto do curso

Bibliotecas	Para quê
para usar IA: LangChain	pedir a um modelo de linguagem, de dentro do programa, que leia, classifique e extraia
para bases de dados: pandas e as de gráfico	ler, filtrar, juntar e resumir tabelas, e desenhar o resultado

Em CDAD 2: conceitos de estatística e as bibliotecas de estatística.

Bibliotecas para usar IA

Insper

Um produto com cliente

O produto da fase 2 vai ter quem use e quem pague por ele.

Com o que o grupo precisa se preocupar aqui que num exercício de aula não precisaria?

Um produto com cliente

RESPOSTA

preocupação	o que o cliente pergunta
custo	quanto vou pagar para rodar a base?
velocidade	em quanto tempo sai o resultado?
experiência de quem usa	quem vai usar consegue rodar e entender?
custo-efetividade	o resultado vale o que custa?

Tudo isso começa em duas decisões: usar um modelo de linguagem ou não e, se sim, qual.

Onde comparar modelos

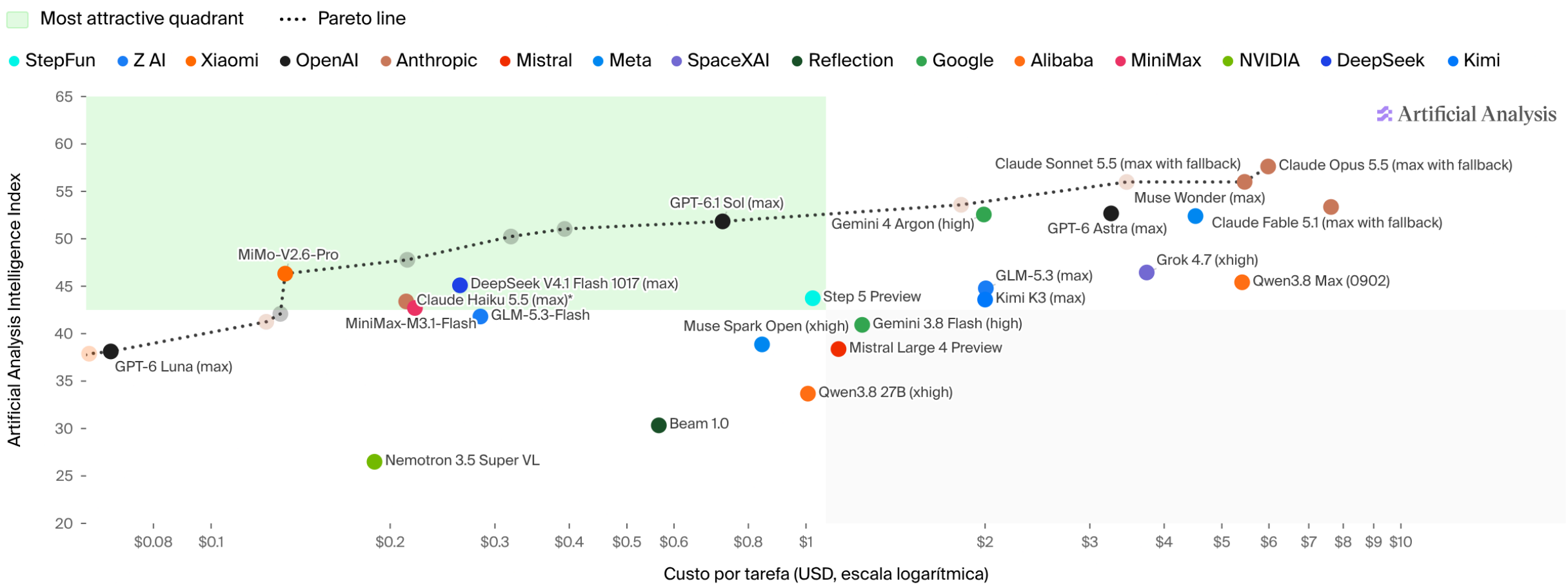
O Artificial Analysis (artificialanalysis.ai) testa os modelos por conta própria e publica três medidas:

medida	como sai
inteligência	um índice de 0 a 100, a partir de baterias de testes
velocidade	tokens por segundo na resposta
preço	dólares por milhão de tokens, ou por tarefa do teste

O índice mede testes gerais, e não a tarefa do grupo. Só testar numa amostra diz se o modelo serve para o produto.

Intelligence Index vs. custo por tarefa

Artificial Analysis Intelligence Index · Weighted average cost (USD) per Artificial Analysis Intelligence Index task



A fronteira de Pareto

Um modelo está na fronteira quando nenhum outro mais barato é mais inteligente. Fora dela, sempre há um modelo melhor pelo mesmo preço, ou mais barato com a mesma inteligência.

No gráfico do Artificial Analysis de 21/10/2026, a fronteira é a linha pontilhada. Ela vai do GPT-6 Luna, perto de US$ 0,07 por tarefa, ao Claude Opus 5.5, perto de US$ 6, passando pelo MiMo-V2.6-Pro, pelo GPT-6.1 Sol e pelo Claude Sonnet 5.5. O quadrado verde é o canto de mais inteligência por menos dinheiro.

É a fronteira de 21/10/2026. Sai modelo novo toda semana: consulte o site no dia da escolha.

Onde rodar

onde	o que dá de graça
Groq, plano gratuito	sem cartão; 1.000 pedidos por dia; é o da demonstração
Google Cloud	US$ 300 por 90 dias, com cartão
AWS	US$ 100, mais até US$ 100 por tarefas, por 12 meses
Azure for Students	US$ 100 por 12 meses, sem cartão

Nas nuvens, os créditos pagam modelos de vários fornecedores. Se o cliente exigir que os dados fiquem no Brasil, pergunte ao provedor, modelo a modelo: a região de São Paulo existe, mas nem todo modelo roda nela.

Cem sentenças para classificar

Insper

Em quantas o pedido foi acolhido?

Uma advogada de consumidor raspou cem sentenças sobre negativa de cobertura de plano de saúde. Ela quer saber em quantas o juiz acolheu o pedido do paciente.

A resposta está no texto de cada sentença, e cada juiz escreve do seu jeito.

Procurar a palavra "procedente"

```
sentencas = [
    "Julgo procedente o pedido.",
    "Julgo improcedente o pedido.",
    "Julgo parcialmente procedente o pedido.",
]
for texto in sentencas:
    print("procedente" in texto)
```

O `in` que vocês usaram com listas também procura um trecho dentro de um texto. O que aparece?

O trecho dentro da palavra

RESPOSTA

```
True
True
True
```

O `in` acha "procedente" em qualquer lugar do texto: dentro de "improcedente" e de "parcialmente procedente" também.

Testar "improcedente" primeiro

```
sentencas = [
    "Julgo improcedente o pedido.",
    "Rejeito o pedido da autora.",
    "O pedido não merece acolhida.",
]
for texto in sentencas:
    if "improcedente" in texto:
        print("improcedente")
    elif "procedente" in texto:
        print("procedente")
    else:
        print("?")
```

O que aparece?

Três sentenças improcedentes, uma reconhecida

RESPOSTA

```
improcedente
?
?
```

As três negam o pedido. Duas dizem isso sem a palavra "improcedente", e cada forma nova de escrever pediria mais um `if`. "Parcialmente procedente" cairia em "procedente". A lista de regras não acaba.

O modelo dentro do programa

Cem sentenças no chat

Uma sentença, vocês colam no chat e perguntam se o pedido foi acolhido.

Como fariam com as cem? E com dez mil?

O programa pergunta ao modelo

RESPOSTA

O programa manda cada sentença ao modelo sozinho, num `for`. Cada envio é um pedido pela internet a um computador do provedor, que devolve uma resposta:

NO COMPUTADOR	**pedido** →	NO PROVEDOR	**resposta** →	DE VOLTA
O programa		O modelo		O texto escrito

O pedido leva o prompt e a chave; a resposta traz o texto. É isso que o chat faz por trás da tela, e é o que se chama de API.

Por que o provedor pede uma chave

Cada resposta custa ao provedor tempo de computador. A chave diz quem está pedindo: a conta de quem paga, ou de quem usa a cota gratuita.

Sem a chave, ou com uma chave que não vale, o provedor recusa o pedido. Com a chave, ele conta quantos pedidos a conta fez e recusa os que passam do limite.

Uma chave só para a turma inteira

Eu crio uma chave e passo para os 55. No mesmo minuto, cada um manda três sentenças ao modelo.

A conta gratuita aceita cerca de 30 pedidos por minuto. O que acontece?

O limite é da conta

RESPOSTA

São 165 pedidos no mesmo minuto. Do trigésimo primeiro em diante, o provedor recusa: a resposta volta com um erro de limite.

E quem tem a chave usa a conta em nome do dono dela. Por isso cada um cria a sua.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

A chave é sua. Não mandem a ninguém, nem para mim.

Onde guardar a chave

O jeito mais rápido é colar a chave no próprio programa:

```
chave = "gsk_..."
```

O programa funciona. O que pode dar errado?

A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário `codigo`.

A chave vai junto com o arquivo

RESPOSTA

O arquivo vai para o repositório, e a chave vai junto. Apagar a linha e enviar de novo não resolve: a versão anterior continua no histórico.

Chave que vazou se apaga no painel da Groq, em **API Keys**, e se troca por outra.

A chave no arquivo .env

#	O que fazer, na pasta `cdad1`
1	No terminal do VS Code: `uv add python-dotenv langchain-groq`
2	Criar o arquivo `.env`, com uma linha só: `GROQ_API_KEY=` e a sua chave colada depois do =
3	Abrir o arquivo `.gitignore` e acrescentar uma linha com `.env`

O `.gitignore` lista o que o Git não registra: o `.env` fica no computador e nunca vai para o GitHub.

Conferir a chave

```python
import os
from dotenv import load_dotenv

load_dotenv()
chave = os.environ["GROQ_API_KEY"]
print(chave[:4])
```

Rodem num notebook da pasta `cdad1`. O que aparece? E se o `.env` não existisse?

os.environ lê o ambiente

RESPOSTA

```
gsk_
```

O `load_dotenv()` leva o que está no `.env` para o ambiente, e `os.environ` é um dicionário com ele. Sem o arquivo, a chave não existe:

```
KeyError: 'GROQ_API_KEY'
```

Só os quatro primeiros caracteres vão para a tela: a chave inteira nunca aparece num print.

A primeira chamada

Insper

Perguntar ao modelo pelo Python

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b",
                  api_key=os.environ["GROQ_API_KEY"])
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
resposta = modelo.invoke(pergunta)
print(resposta.content)
```

Rodem. O que aparece?

Objeto, método e atributo

RESPOSTA

Aparece o texto que o modelo escreveu, e cada um viu o seu.

No código	O que é
`ChatGroq(...)`	cria o objeto do modelo
`modelo.invoke(...)`	o método que manda o pedido
`resposta`	um objeto, com o texto e os tokens
`resposta.content`	o atributo com o texto

O pedido e a resposta, passo a passo

Animação no navegador: `animacao-api-llm.html`, cenário `ambiente`.

Agora, praticar

#	Com o programa da primeira chamada
1	Troquem a pergunta por uma das sentenças da lista e peçam uma palavra só: procedente, improcedente ou parcial
2	Repitam com as outras duas sentenças
3	Comparem com o vizinho: as respostas batem?

Montar o prompt

Uma instrução de três linhas

```
texto = "Rejeito o pedido da autora."
prompt = """Leia a sentença e responda com uma palavra:
PROCEDENTE, IMPROCEDENTE ou PARCIAL.

Sentença: {texto}"""
print(prompt)
```

O prompt vai receber cada sentença do lote. O que aparece?

As chaves ficaram no texto

RESPOSTA

```
Leia a sentença e responda com uma palavra:
PROCEDENTE, IMPROCEDENTE ou PARCIAL.

Sentença: {texto}
```

Nenhum erro, e a sentença não entrou. O modelo receberia `{texto}` e responderia sobre nada.

Sem o f

Animação no navegador: `animacao-prompt.html`, cenário `sem-f`.

Um f antes das aspas

```
texto = "Rejeito o pedido da autora."
prompt = f"""Leia a sentença e responda com uma palavra:
PROCEDENTE, IMPROCEDENTE ou PARCIAL.

Sentença: {texto}"""
print(prompt)
```

A única diferença é o `f` antes das três aspas. O que aparece?

f-string

RESPOSTA

```
Leia a sentença e responda com uma palavra:
PROCEDENTE, IMPROCEDENTE ou PARCIAL.

Sentença: Rejeito o pedido da autora.
```

Com o `f`, o que está entre chaves é trocado pelo valor da variável. É a f-string, e com três aspas ela pode ter várias linhas.

Com o f

Animação no navegador: `animacao-prompt.html`, cenário `com-f`.

O prompt dentro do for

```
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b")
for texto in ["Rejeito o pedido.", "O pedido não procede."]:
    prompt = f"""Responda PROCEDENTE ou IMPROCEDENTE.
Sentença: {texto}"""
    resposta = modelo.invoke(prompt)
    print(resposta.content)
```

Quantos pedidos saem para o modelo? E onde foi parar o `api_key=`?

Um pedido por sentença

RESPOSTA

Dois pedidos, um por volta do `for`, cada um com a sua sentença dentro do prompt. Com cem sentenças, cem pedidos.

O `api_key` é um parâmetro opcional. Sem ele, o `ChatGroq` usa o valor padrão: procura `GROQ_API_KEY` no ambiente.

A mesma pergunta, três vezes

Rodem três vezes seguidas a célula da primeira chamada, a de "Rejeito o pedido. Procedente ou improcedente?", e anotem as respostas.

As três saíram iguais, palavra por palavra?

O mesmo pedido, outro texto

RESPOSTA

Em geral, não: a frase muda de uma rodada para outra, mesmo com o modelo entendendo a sentença do mesmo jeito.

Um `if resposta.content == "Improcedente.":` erraria em parte das rodadas.

Temis: como venceu a fase 1

Insper

Temis, vencedora da fase 1



Um assistente que lê o aviso de andamento do processo no e-mail, calcula o prazo e põe na agenda.

O grupo conta como se dividiu, que referências buscou e que ferramentas usou. Depois, as perguntas são de vocês.

Fase 2: banca, demonstração e nota

Insper

A banca da fase 2

Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

A banca da fase 2

Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

O que costuma pesar

Em bancas de tecnologia: se o produto funciona de verdade; se aguenta erro; se o código é legível; se o grupo sabe explicar o que o assistente faz.

A demonstração

Quando	O quê
Até 17/11	um vídeo do produto funcionando, enviado com o endereço do repositório
Em 18/11	o pitch, como na fase 1: slides e demonstração do jeito que o grupo preferir

A tier list com dez grupos

Faixa	S	A	B	C	D	F
Vagas	1	2	2	2	2	1
Pontos	15	12	9	6	3	1

As mesmas faixas e os mesmos pontos da fase 1. São dez vagas para dez grupos: nenhuma faixa leva um grupo a mais.

A nota da fase 2

Posição	Nota
1ª e 2ª	1,5
3ª a 6ª	1,4; 1,3; 1,2; 1,1
7ª a 10ª	de 0,1 a 1,0, pelo repositório

Como o modelo escreve

O texto vira tokens

Animação no navegador: `animacao-tokens.html`, cenário `quebrar`.

A próxima palavra de uma ementa

```
EMENTA: Apelação cível. Plano de saúde.
Negativa de cobertura. Recurso ...
```

Que palavra vem depois de "Recurso"? Deem também uma chance, em porcentagem, para o seu palpite.

Um token de cada vez

Animação no navegador: `animacao-tokens.html`, cenário `gerar`.

Temperatura

RESPOSTA

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b")
print(modelo.temperature)
```

```
0.7
```

Com temperatura acima de zero, o modelo sorteia entre os tokens prováveis, e o mesmo pedido pode dar outro texto. Com `temperature=0`, ele pega sempre o mais provável.

Pedir o formato não garante o formato

"Responda com uma palavra" deixa a palavra muito provável, e não certa. O modelo escreve o que é provável, token a token.

Por isso a resposta do modelo se confere: numa amostra das cem sentenças, alguém lê e compara.

Como o modelo foi treinado

De onde vêm as probabilidades

O modelo nunca leu as sentenças de vocês.

Como ele sabe que, depois de "Recurso", costuma vir "extraordinário"?

Pré-treino

RESPOSTA

Ele leu uma quantidade enorme de texto, cobrindo cada próximo token e tentando adivinhá-lo. A cada erro, os números da rede neural, chamados de pesos, são ajustados um pouco.

Repetido trilhões de vezes, sobra um modelo que sabe o que costuma vir depois do quê, inclusive em texto de tribunal.

Adivinhar o próximo token, errar, ajustar

Animação proposta: `animacao-treino.html`, cenário `pre-treino`.

Uma frase de ementa com o último token coberto; o modelo dá probabilidades, o token real aparece, e os pesos mudam de cor conforme o erro. Na volta seguinte, a probabilidade do token certo sobe.

O treino do gpt-oss-120b

trilhões

de tokens de texto lidos no pré-treino, com foco em ciência, programação e conhecimento geral, e corte em junho de 2024

O treino do gpt-oss-120b

trilhões

de tokens de texto lidos no pré-treino, com foco em ciência, programação e conhecimento geral, e corte em junho de 2024

2,1 milhões

de horas de GPU para treinar o modelo de cerca de 120 bilhões de pesos: 240 anos numa GPU só

O treino do gpt-oss-120b

trilhões

de tokens de texto lidos no pré-treino, com foco em ciência, programação e conhecimento geral, e corte em junho de 2024

2,1 milhões

de horas de GPU para treinar o modelo de cerca de 120 bilhões de pesos: 240 anos numa GPU só

1 GPU

de 80 GB basta para rodar o modelo pronto; treinar pediu muitas delas juntas, num data center

Um modelo que só passou pelo pré-treino

Ele recebe este texto:

```
Responda com uma palavra: PROCEDENTE ou IMPROCEDENTE.
Sentença: Rejeito o pedido da autora.
```

O que ele faz com isso?

Pós-treino

RESPOSTA

Ele continua o texto, como continuou a ementa: pode escrever outra sentença ou outra pergunta. Nada no pré-treino o ensinou a obedecer.

Obedecer vem do pós-treino: o modelo resolve pedidos e problemas de muitos tipos, e o treino reforça as respostas boas. É o que transforma o modelo que completa no assistente do chat.

O mesmo pedido, antes e depois do pós-treino

Animação proposta: `animacao-treino.html`, cenário `pos-treino`.

O mesmo prompt entra nos dois modelos: o de pré-treino continua o texto, o de pós-treino responde. Entre um e outro, pedidos com respostas, e as boas reforçadas.

O que o treino explica

O que vocês veem	De onde vem
Escreve em português de tribunal	o pré-treino, que leu esse texto
Obedece "responda com uma palavra"	o pós-treino
Não sabe de lei ou decisão de julho de 2024 em diante	os dados do pré-treino param em junho de 2024
Dá um número de súmula com a mesma segurança, exista ela ou não	ele escreve o token provável, sem consultar fonte nenhuma

O que cabe num pedido

Insper

Dois pedidos seguidos

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b")
modelo.invoke("A autora se chama Ana Lima.")
resposta = modelo.invoke("Como se chama a autora?")
print(resposta.content)
```

O modelo responde "Ana Lima"?

Cada pedido chega sozinho

RESPOSTA

O segundo pedido leva só isto, e nada do primeiro:

```
[{"role": "user", "content": "Como se chama a autora?"}]
```

O modelo não guarda nada entre um pedido e outro. Quem quer uma conversa reenvia a conversa.

Uma lista de mensagens

```
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b")
historico = [("user", "A autora se chama Ana Lima.")]
resposta = modelo.invoke(historico)
historico.append(("assistant", resposta.content))
historico.append(("user", "Como se chama a autora?"))
resposta = modelo.invoke(historico)
print(len(historico))
```

Quantas mensagens o histórico tem no fim? O que vai no segundo pedido?

O histórico vai inteiro, a cada pedido

RESPOSTA

```
3
```

Três mensagens, e as três vão no segundo pedido: a de vocês (`"user"`), a resposta do modelo (`"assistant"`) e a pergunta nova. A cada volta da conversa, o pedido fica maior.

A janela de contexto enchendo

Animação proposta: `animacao-contexto.html`, cenário `conversa`.

A cada volta da conversa, a lista de mensagens cresce e é reenviada inteira; uma barra com o número de tokens enche em direção ao limite da janela.

A janela de contexto

Tudo o que vai num pedido divide um espaço só: instrução, histórico, documentos e a resposta que vem. É a janela de contexto, medida em tokens.

Medida	Tokens
Janela do `openai/gpt-oss-120b`	131.072
Texto de tribunal, por palavra	cerca de 1,65
Uma página de resultados do TJSP, com dez decisões	cerca de 24.600

Uma página inteira num pedido só

A página de resultados do TJSP tem cerca de 24.600 tokens. A janela do modelo tem 131.072. A conta gratuita aceita cerca de 8.000 tokens por minuto.

Mandamos a página inteira num pedido só. Cabe na janela? A Groq responde?

Dois limites diferentes

RESPOSTA

Limite	De quem	Aqui
Janela de contexto	do modelo	cabe: 24.600 é menos que 131.072
Tokens por minuto	da conta	não passa: 24.600 é mais que 8.000, e a Groq recusa

`resposta.usage_metadata` diz quantos tokens cada pedido gastou: `input_tokens`, `output_tokens` e `total_tokens`.

Um documento grande na janela

Animação proposta: `animacao-contexto.html`, cenário `documento`.

A página do TJSP entra na janela de 131.072 tokens e ocupa um quinto dela; ao lado, o contador do minuto da conta transborda em 8.000, e o pedido volta recusado.

LangChain

O mesmo pedido pela biblioteca da Groq

```python
from dotenv import load_dotenv
from groq import Groq
load_dotenv()
cliente = Groq()
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
resposta = cliente.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": pergunta}],
)
print(resposta.choices[0].message.content)
```

Onde está o texto da resposta? Comparem com o `resposta.content` da primeira chamada.

Cada provedor, um caminho até o texto

RESPOSTA

Na biblioteca da Groq, o texto da resposta fica em:

```
resposta.choices[0].message.content
```

A biblioteca de cada empresa tem nomes e caminhos próprios, e trocar de empresa obrigaria a reescrever o programa.

O LangChain põe todas atrás das mesmas peças: o modelo, as mensagens, o `invoke` e o `.content`.

O mesmo invoke para três provedores

Animação proposta: `animacao-provedores.html`, cenário `trocar`.

O mesmo `modelo.invoke(...)` sai para a Groq, a OpenAI e o Google; cada pedido volta num JSON de formato diferente, e o LangChain entrega os três como o mesmo `.content`.

Trocar de provedor é trocar dois textos

```python
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
load_dotenv()
modelo = init_chat_model("openai/gpt-oss-120b",
                         model_provider="groq")
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
resposta = modelo.invoke(pergunta)
print(resposta.content)
```

O `model_provider` diz de que empresa é o modelo, e o primeiro texto, qual modelo dela.

Os nomes dos provedores

model_provider	Biblioteca a instalar
"groq"	langchain-groq
"openai"	langchain-openai
"google_genai"	langchain-google-genai
"anthropic"	langchain-anthropic

As peças do LangChain desta aula

Peça	No código
O modelo	`ChatGroq(...)` ou `init_chat_model(...)`
As mensagens	uma lista de pares: `"system"`, `"user"`, `"assistant"`
O pedido	`modelo.invoke(...)`
A resposta	`.content` com o texto, `.usage_metadata` com os tokens

Na próxima aula

Como o modelo deixa de só responder e passa a agir: um programa que dá ferramentas ao modelo, roda em loop até cumprir o objetivo e pede permissão antes de mexer no que importa.

Até o início dela, quarta, 28/10: a sua chave da Groq funcionando, com a primeira chamada respondendo no seu computador.